Reading Round-Up Dashboard

Handover Documentation

Table of Contents

1. Executive Summary

The Reading Round-Up Dashboard helps Claire Adler Luxury PR collect relevant articles, create concise summaries, produce reading-roundup PDFs, email a scheduled weekly roundup, analyse trends, generate client pitch ideas, and prepare client coverage reports. It also manages publication source lists and checks whether those sources are still healthy.

The frontend is the website used by the team. The FastAPI backend handles login and all privileged work. GitHub Actions runs long jobs. Google Sheets stores articles, configuration, results, and status information. This separation is important because GitHub tokens, Google service-account credentials, passwords, and paid API keys must never be placed in browser code.

The current version uses Qwen models for grounded article summaries, relevance work, topic keyword weighting, and client pitch generation. The current roundup aims for approximately 50 articles, applies publication-balance and quality rules, and backfills reserve articles when later stages remove candidates.

2. Mental Model of the System

Think of the system as a dashboard in front of several controlled background processes.

- The dashboard is the control surface. It collects user choices and displays results.
- The backend is the gatekeeper. It authenticates users, validates requests, reads and writes operational data, and dispatches GitHub workflows.
- GitHub Actions is the worker. It runs collection, summarisation, analysis, email, pitch, coverage, and source-health jobs that may take too long for a normal web request.
- Google Sheets is the shared operational database. It is readable by non-technical staff and is used by both the backend and workflows.
- GitHub artifacts carry generated PDFs and diagnostic files. Render hosts the backend and GitHub Pages hosts the frontend.

Normal article-roundup flow

1. A user signs in, chooses a topic/source list, optionally supplies keyword overrides, and starts the pipeline.
2. The frontend sends the request to the backend.
3. The backend checks authentication, active-run state, and cooldown rules, then dispatches collect-articles.yml.
4. The workflow prepares credentials, generates any missing topic keyword weights, collects and filters articles, creates Qwen summaries, saves results to Google Sheets, and generates a PDF.
5. The workflow uploads the PDF as an artifact. The dashboard polls status and makes the result available to the user.

Other flows

Trend analysis, client pitches, and client coverage follow the same broad pattern: the dashboard sends an authenticated request, the backend dispatches a workflow with a run or job ID, the workflow writes results, and the dashboard polls until the requested result is ready. Client coverage is deliberately staged so a person can review articles and countries before the final PDF is produced.

3. Technical Design

3.1 Frontend

The frontend is in frontend/. It uses React 19, Vite 7, Tailwind CSS 4, Axios, date-fns, and lucide-react. frontend/src/App.jsx defines five main views: Article Summaries, Weekly Email, Trend Analysis, Client Pitch Generator, and Client Coverage.

frontend/src/services/githubAPI.js is the main backend client despite its older filename. It handles login, token refresh, pipeline status, artifacts, stars, topic/source configuration, trends, client pitches, and coverage jobs. frontend/src/services/googleSheetsAPI.js provides read-only browser access for article display using a restricted Google API key.

Important frontend files include SummariesView.jsx, WeeklyEmailSettingsView.jsx, TrendAnalysisView.jsx, ClientPitchGeneratorView.jsx, ClientCoverageScannerView.jsx, ManageTopicsPage.jsx, LoginScreen.jsx, LoadingScreen.jsx, and Navigation.jsx.

3.2 Backend

The backend is in backend/. backend/trigger_service.py is the FastAPI entry point and security boundary. It provides login, refresh and logout; article pipeline trigger/status/artifact endpoints; weekly-email configuration; stars; topic and source-list management; trend endpoints; client and pitch endpoints; client coverage job endpoints; and source metadata validation endpoints.

backend/pipeline_runner.py coordinates the main article pipeline. backend/AgentCollector.py discovers and extracts articles. backend/AgentSumm.py creates grounded Qwen summaries and finalises author data. backend/google_storage.py reads and writes Google Sheets and contains older Drive helpers. backend/PDFGenerator.py creates the reading-roundup PDF.

Quality and selection logic is separated into roundup_selection.py, article_quality.py, article_relevance_classifier.py, and qwen_runtime.py. Topic keyword policies are generated by run_topic_keyword_weighting.py and read from Topic Config.

3.3 Current workflows

- collect-articles.yml: manual main roundup; Python 3.10; 240-minute job timeout; one retry around a 230-minute command timeout; Qwen 2.5 3B GGUF; PDF artifact retained for 90 days.
- weekly-reading-roundup.yml: scheduled Mondays at 09:17 UTC and manually runnable; loads weekly settings, runs the roundup, uploads the PDF, and emails it using SMTP secrets.
- trend-analysis.yml: manual; Python 3.11; compares a selected window with a historical baseline and writes Trend Signals.
- client-pitch-generator.yml: manual; Python 3.11; generates up to eight client pitch ideas using Qwen 2.5 1.5B and selected evidence.

- client-coverage-report.yml: manual staged workflow; Python 3.11; discover, verify, country, and finalize actions; uses SerpApi and ZenRows where required.
- biweekly-source-health.yml: runs at 09:23 Europe/London on Mondays in even ISO weeks and can run manually; diagnoses sources, applies safe repairs, quarantines repeat failures, attempts recovery, and uploads per-topic reports.
- deploy-frontend.yml: builds with Node 20 and deploys frontend changes from master or App-revamp to GitHub Pages.
- test-email.yml: manual utility for checking email configuration.

3.4 Scraping and source discovery

AgentCollector.py loads active sources from Source Lists. It uses sitemap and RSS discovery, URL cleaning, deduplication, keyword scoring, publication limits, and full-text extraction. Request handling can use curl_cffi, cloudscraper, or requests. User-agent rotation, delays, fallbacks, and source-level failure handling reduce the chance that one blocked publication stops the full run.

A valid RSS feed or sitemap does not guarantee usable article pages. Paywalls, placeholders, irrelevant archive pages, dynamic content, and anti-bot systems can still prevent extraction. Source validation and actual collection results must therefore be reviewed together.

3.5 Qwen summarisation and selection

Production roundup workflows set SUMMARY_MODEL_REPO to Qwen/Qwen2.5-3B-Instruct-GGUF and SUMMARY_MODEL_FILE to qwen2.5-3b-instruct-q4_k_m.gguf. llama-cpp-python runs the local GGUF model. A topic can supply a custom summary prompt through Topic Config. Summaries are expected to remain grounded in the extracted article rather than add unsupported facts.

Default roundup controls aim for a balanced, useful document: minimum goal 40 articles, target 50, up to 60 candidates, soft publication cap 5, hard cap 10, reserve pool 10, summary-failure buffer 10, and lifestyle reserve cap 5. Luxury relevance, article quality, duplicate control, publication normalisation, royal-story limits, and reserve backfill are applied before the final output.

3.6 Trend analysis

run_trend_analysis.py filters stored articles to hosts in the selected source list, compares a selected current window with a baseline, and writes ranked rows to Trend Signals. Current workflow defaults are minimum 2 mentions, minimum lift 1.2, minimum 1 publication, and top 25 results. A NO_TRENDS row is normal when no signal passes the thresholds.

3.7 Client pitch generator

Clients are stored with a name, description, and topic. The user can generate pitches from Trend Signals, recent coverage, or Auto mode, which prefers trends and falls back to recent coverage. The GitHub workflow uses Qwen/Qwen2.5-1.5B-Instruct and writes results to Client Pitches. The dashboard polls by pitch run ID.

3.8 Client coverage

Client coverage uses persistent jobs and four stages: discover, verify, country, and finalize. Users enter a report title, mention terms, optional backlink domains, search queries, and a date range. Search results are reviewed manually; publication names can be corrected; country results can be confirmed, overridden, or marked not applicable; and only then is the report finalised.

SerpApi is required for discovery. ZenRows is supplied to the finalisation stage for publication traffic work. Coverage-specific dependencies are isolated in requirements-coverage.txt. The final PDF is created by client_coverage_pdf.py and downloaded through the backend.

3.9 Security boundary

Anything beginning with VITE_ is shipped to the browser. Never place a GitHub token, service-account JSON, app password, SMTP password, SerpApi key, or ZenRows key in a VITE_ variable. Privileged values belong in Render environment variables or GitHub Actions repository secrets. The browser Google key must be restricted to the required API and approved referrers.

The backend uses JWT access tokens and refresh cookies. Important settings include APP_LOGIN_PASSWORD, JWT_SECRET, JWT_EXPIRES_SECONDS, REFRESH_EXPIRES_SECONDS, REFRESH_COOKIE_SECURE, and REFRESH_COOKIE_SAMESITE. Keep CORS restricted to approved local and deployed origins.

4. Article Pipeline Database

The Article Pipeline Database is a Google Sheets workbook used as both the article database and the lightweight operations database. A Google service account connects through backend/google_storage.py and related modules. GOOGLE_SHEET_ID selects the workbook; GOOGLE_CREDENTIALS provides the service-account JSON in Render and GitHub Actions.

Important worksheets

- Articles: final article records, including title, URL, publication, journalist, summary, date information, and score.
- Metadata: latest pipeline and workflow state, progress, counts, run IDs, URLs, and related status values.
- Source Lists: publications grouped by list/topic, discovery URLs, active state, and maintenance information.
- Topic Config: topic names, keyword policies, and custom summary prompts.
- Source Validation Details: results from metadata validation.
- Source Health History: scheduled diagnostic results, repairs, quarantine, and recovery history.
- Trend Signals: ranked trend output by run and window.
- Starred Summaries: articles saved by users for follow-up.
- Clients and Client Pitches: client descriptions/topics and generated pitch ideas.
- Client Coverage Reports and related job data: persistent coverage state and report rows.

Article rows are appended so history is preserved. Do not casually rename worksheets or columns: backend code often expects exact names. Before restructuring the workbook, search the codebase for the sheet and header names and test every affected workflow.

5. Local Setup

Local setup is for understanding the application and testing small changes. Production secrets should not be copied into frontend files or committed to the repository.

Backend

From App-revamp/backend:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn trigger_service:app --reload
```

On macOS or Linux, activate with source .venv/bin/activate. trigger_service.py requires its core environment variables at import time, including GitHub owner/repository/ref/token and app authentication values. Google-backed features also require GOOGLE_SHEET_ID and GOOGLE_CREDENTIALS.

Frontend

From App-revamp/frontend:

```
npm install
npm run dev
```

Create a local frontend environment file only when needed. The normal values are VITE_PIPELINE_API_BASE, VITE_GOOGLE_SHEET_ID, and a restricted VITE_GOOGLE_API_KEY. Do not add VITE_GITHUB_TOKEN.

Basic local checks

1. Confirm the backend starts without missing-variable or import errors.
2. Open the frontend and test login.
3. Confirm /pipeline/status responds through the frontend API client.
4. Load one source list and recent article results.
5. Use browser developer tools to confirm the frontend calls VITE_PIPELINE_API_BASE rather than GitHub directly.

6. Usage Guide

Article Summaries

1. Sign in and open Article Summaries.
2. Choose the correct topic or source list.
3. Optionally enter a keyword override, then start the pipeline.
4. Leave the run active. It can take a long time; the workflow permits up to four hours.
5. When complete, review the summaries, star useful articles, and download the roundup artifact.

Weekly Email

Use Weekly Email Settings to control the saved configuration. The scheduled job runs Mondays at 09:17 UTC. Confirm the selected topic/list and recipients before relying on the next scheduled send. UTC does not remain the same UK local time throughout the year.

Trend Analysis

Choose a source list and current week, current month, or custom window. Start the run and wait for its status. If there are no results, verify that the selected list has active hosts and that Articles contains matching URLs and dates.

Client Pitch Generator

Select or add a client, make sure the client has a topic and useful description, choose the evidence mode, choose 1-8 ideas, and generate. Auto is normally the safest mode. Review ideas before using them; generated text is a draft, not final client advice.

Client Coverage

1. Enter the report title, mention terms, search queries, and date range.
2. Run discovery. Use focused searches and exact-name quotes.
3. Review candidates and accept only genuine coverage. Correct publication names where necessary.
4. Run or complete the country check. Confirm correct results and override unresolved items.
5. Finalize only after the review queues are empty, then download and inspect the PDF.

For simple, non-technical instructions on coverage searches and changing the SerpApi or ZenRows key, use operations-guide.pdf.

7. Google Cloud and Service Account Monitoring

The Google service account allows the backend and workflows to read and write the Article Pipeline Database without using a personal Google login. The service account should remain enabled, its key should remain valid, and its email should retain Editor access to the workbook.

Where to check

1. In Google Cloud Console, open the project used by the application.
2. Go to IAM & Admin, then Service Accounts, and confirm the Reading Round-Up service account is enabled.
3. Open the Google Sheet's sharing settings and confirm the service-account email has Editor access.
4. In APIs & Services, confirm Google Sheets API is enabled. Keep Google Drive API enabled while Drive helper code or related operations remain in the repository.
5. Confirm GOOGLE_CREDENTIALS and GOOGLE_SHEET_ID are current in GitHub repository secrets and Render environment variables.

Credential rotation

If a service-account key is rotated, update the complete JSON everywhere the backend or workflows run. Do not paste it into documentation, chat, tickets, or source files. After updating it, test a low-risk read and write operation such as loading source lists and running source validation with repairs disabled.

Common symptoms

- The spreadsheet cannot be opened or a worksheet cannot be found.
- Article rows, trend rows, pitches, or coverage data are not written.
- Source metadata or health jobs fail immediately.
- The dashboard cannot load backend-provided lists or results.

8. Maintenance and Operations

After each weekly roundup

- Confirm the workflow, PDF artifact, and email step succeeded.
- Check article count. The target is about 50 and the minimum goal is 40, but source availability and quality rules can reduce it.
- Read a small sample of summaries for grounding, clarity, correct publication, and sensible author information.
- Investigate sudden count changes rather than assuming the dashboard is broken.

Every two weeks

Review the Biweekly Source Health Diagnostic workflow summary and artifact. Check sources that were repaired, quarantined, recovered, or left unresolved. Use the per-topic report to see whether one source list is weakening.

Monthly

- Review GitHub Actions duration and failures, Render logs, and Google Sheet growth.
- Check SerpApi and ZenRows usage and billing limits.
- Review source-list accuracy, trend quality, client pitches, and coverage reports.
- Run frontend lint/build and relevant backend tests before dependency or deployment changes.

Quarterly and after staff changes

Audit access to GitHub, Render, Google Cloud, the Sheet, SMTP, SerpApi, and ZenRows. Remove access that is no longer needed and rotate exposed or shared credentials. Do not wait for a scheduled rotation after a key has been placed in a document or message.

9. Troubleshooting Guide

No GitHub run appears

Check the backend response and Render logs. Verify login/authentication, cooldown or active-run state, GITHUB_OWNER, GITHUB_REPO, GITHUB_TOKEN, GITHUB_WORKFLOW, COVERAGE_WORKFLOW where used, and GITHUB_REF. Confirm the workflow exists on that ref.

A workflow starts but fails

Open the failed GitHub Actions step. Common causes are dependency installation, model download, missing or invalid Google credentials, incorrect sheet names, extraction failures, SerpApi quota, ZenRows errors, SMTP settings, or artifact generation.

Roundup has fewer than 40 articles

Review the source-health report, source activity, extraction errors, topic weights, relevance and royal filters, publication caps, reserve exhaustion, and summary failures. A smaller high-quality roundup may be correct, but a sharp unexplained drop needs investigation.

Articles are in Sheets but not the dashboard

Check VITE_PIPELINE_API_BASE, the browser Google key restrictions, CORS, authentication, frontend console/network errors, sheet headers, and date parsing. If the rows are absent from Sheets, inspect collection, summarisation, and storage instead.

Trend results are empty

Check the source list, active hosts, selected dates, baseline length, article volume, and host matching. A NO_TRENDS sentinel means the run completed but nothing passed the thresholds.

Pitch results are empty

Check the client's topic and description, evidence mode, Trend Signals availability, recent Articles data, pitch run ID, and workflow status.

Coverage fails

For discovery, check SERPAPI_API_KEY, provider status, quota, date range, and search queries. For finalisation/traffic, check ZENROWS_API_KEY. For stalled jobs, check the job ID, active action, GitHub run, uploaded stage artifact, pending review rows, and backend state reconciliation.

Authentication or cookie problems

Verify APP_LOGIN_PASSWORD, JWT_SECRET, refresh-cookie security and SameSite settings, CORS origins, and the browser's stored session. After deployment changes, sign out or clear site cookies and sign in again.

10. Known Limitations

- Scraping remains vulnerable to paywalls, bot protection, dynamic rendering, changing feed paths, and changing page structure.
- Source validation can prove that discovery endpoints work but cannot guarantee every article page is relevant and extractable.
- Author extraction is imperfect because publications represent bylines in inconsistent ways.
- Progress is clearer than in the prototype but is still mostly workflow/stage based rather than live article-by-article reporting.
- Qwen models run on GitHub Actions CPU and may be slow or fail because of model downloads, native dependencies, network problems, or memory limits.

- The repository currently mixes Python 3.10 and 3.11 across workflows. Standardisation requires testing model and native dependencies first.
- GitHub artifacts have finite retention. Principal report artifacts are generally retained for 90 days and should be archived elsewhere when longer retention is required.
- Trend analysis uses text features and thresholds rather than full semantic clustering. BERTopic is installed but is not the active production trend method.
- Generated summaries and pitch ideas still require human review. A successful model run does not guarantee editorial suitability.

11. Next Steps

Immediate

- Rotate any credential that has appeared in an earlier document, message, screenshot, or repository history, and keep new values only in approved secret stores.
- Confirm Render GITHUB_REF and the frontend deployment branches match the team's active production branch strategy.
- Run a full scheduled-roundup check, including email delivery and PDF review, after handover.
- Review the newest source-health artifact and resolve any quarantined or unresolved high-priority publications.
- Confirm all non-technical users can follow operations-guide.pdf to run coverage and replace SerpApi/ZenRows secrets.

Near term

- Standardise Python versions only after testing Qwen/llama.cpp, scraping, and coverage dependencies.
- Improve live progress with source-level and article-level updates that survive backend restarts.
- Make dashboard publication counts and runtime guidance derive from current source lists and recent runs.
- Add end-to-end tests for scheduled email, deployed authentication/cookies, source-health writeback, client pitch runs, and complete coverage jobs.
- Create an archive policy for report artifacts and older Google Sheet rows.

12. Future Projects

- Improve journalist/byline extraction with publication-specific rules or a named-entity model.
- Train or tune topic-specific relevance models to complement or replace keyword-weight rules.
- Add semantic topic clustering to trend analysis and compare its usefulness with the current transparent threshold approach.
- Merge the trend signals and clietn pitch generator into one workflow that uses the trend signals to create the pitches. And, improve/ fine-tune the llm used to create the pitches.
- Turn approved trend signals and pitch ideas into presentation-ready pitch decks with a required human approval step.

- Build a client outreach email drafter that uses approved client facts and selected evidence without sending messages automatically.
- Move persistent workflow and job state to a database or shared store if Sheets and local state become unreliable at larger scale.
- Improve long-term report storage, access control, and search beyond temporary GitHub artifacts.
- Continue reviewing Thoughts on Future Projects.pdf after the immediate operational priorities are stable.